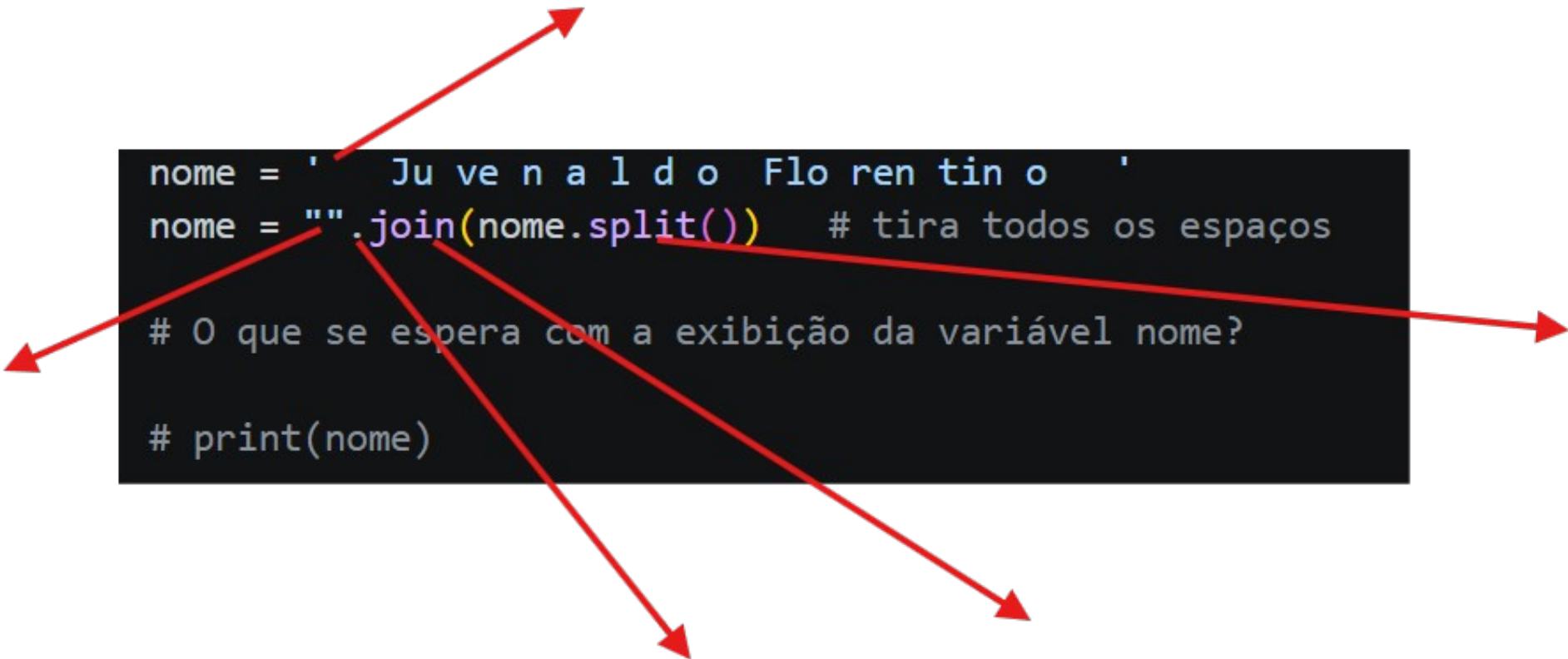


```
nome = ' Ju ve n a l d o Flo ren tin o '
```

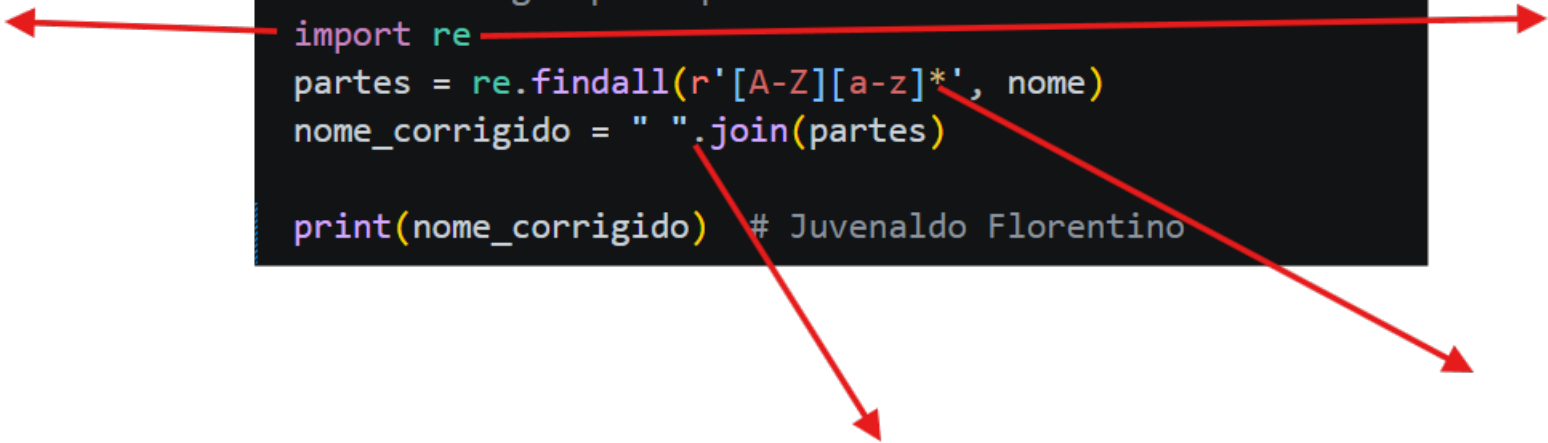
nome = `"".join(nome.split())` # tira todos os espaços

O que se espera com a exibição da variável nome?

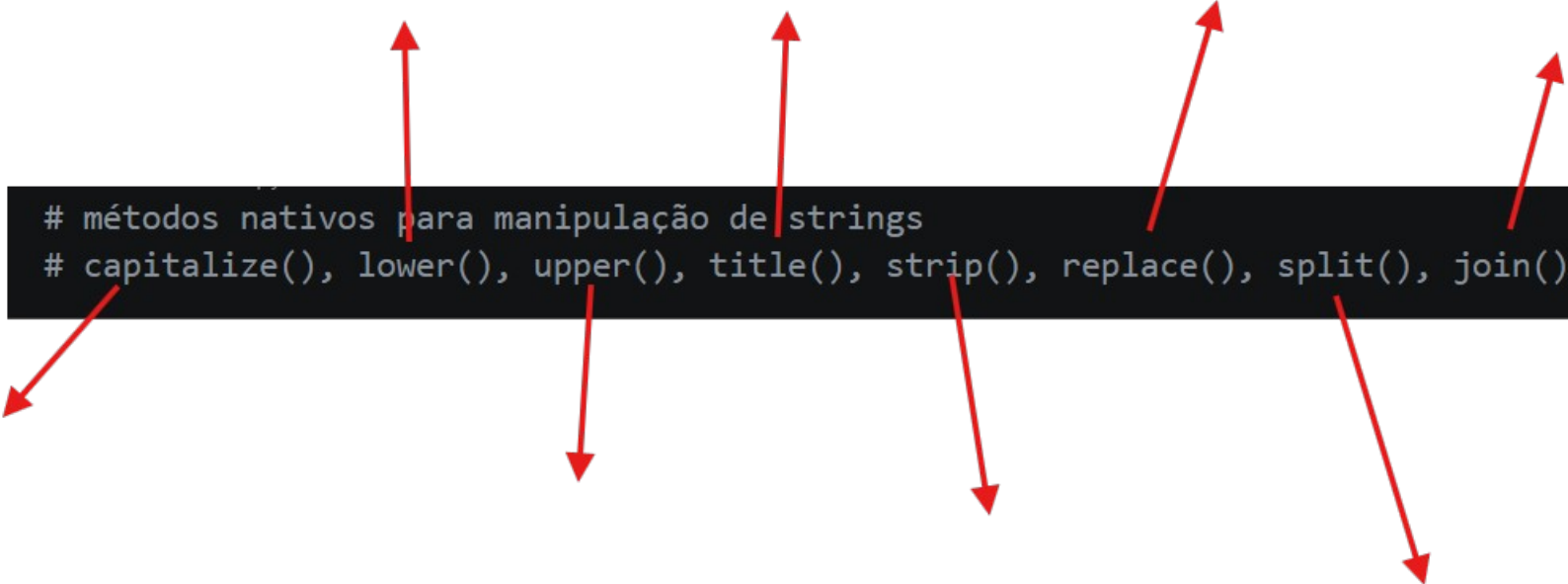
print(nome)

A diagram consisting of five red arrows originates from the code block. One arrow points from the first single quote of the first line to the top right. Another points from the second single quote of the first line to the top left. A third arrow points from the opening double quote of the second line to the bottom left. A fourth arrow points from the `split()` method call in the second line to the bottom center. The final arrow points from the `join()` method call in the second line to the bottom right.

```
nome = '    Ju ve n a l d o  Flo ren tin o    '  
nome = "".join(nome.split()) # tira todos os espaços  
  
# O que se espera com a exibição da variável nome?  
  
# print(nome)  
  
# Separar em palavras: "Juvenaldo" + "Florentino"  
# usando regex para quebrar entre maiúsculas  
import re  
partes = re.findall(r'[A-Z][a-z]*', nome)  
nome_corrigido = " ".join(partes)  
  
print(nome_corrigido) # Juvenaldo Florentino
```



```
# métodos nativos para manipulação de strings  
# capitalize(), lower(), upper(), title(), strip(), replace(), split(), join()
```



A diagram consisting of nine red arrows. Five arrows point upwards from the top edge of the code block to various positions above it. Four arrows point downwards from the bottom edge of the code block to various positions below it. The arrows are distributed across the width of the code block, with some pointing towards the left and others towards the right.



```
...  
.lower() → converte para minúsculas  
.upper() → converte para maiúsculas  
.capitalize() → deixa só a primeira letra em maiúscula  
.title() → coloca a primeira letra de cada palavra em maiúscula  
.swapcase() → inverte maiúsculas ↔ minúsculas  
.casefold() → versão mais agressiva de lower() (bom para comparações)  
...
```

Variaveis string - função len →

```
nome = "Heitor"
```

✓ 0.0s

```
print(len(nome)) # Retorna o tamanho da string
```

✓ 0.0s

Manipulacao de string

```
alfabeto = "abcdefghijklmnopqrstuvwxyz"
```

✓ 0.0s

```
print(alfabeto[0]) # Retorna o primeiro caractere da string
```



Concatenação

```
letras = "ABDE"
```



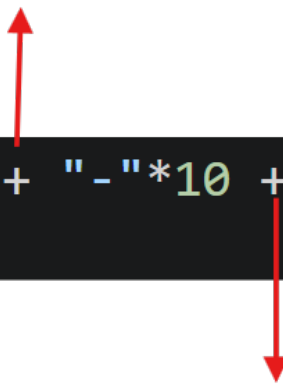
✓ 0.0s

```
print(letras + "FGH") # Concatena duas strings
```





```
print(letras + "F" * 5) # Concatena a string com "F" repetido 5 vezes
```

```
print("X" + "-"*10 + "X") # Cria uma linha com "X" nas extremidades e "-" no  
meio
```

The image shows a line of Python code on a dark background. The code is `print("X" + "-"*10 + "X")` followed by a comment `# Cria uma linha com "X" nas extremidades e "-" no meio`. Two red arrows are drawn: one points upwards from the first '+' sign, and the other points downwards from the second '+' sign.

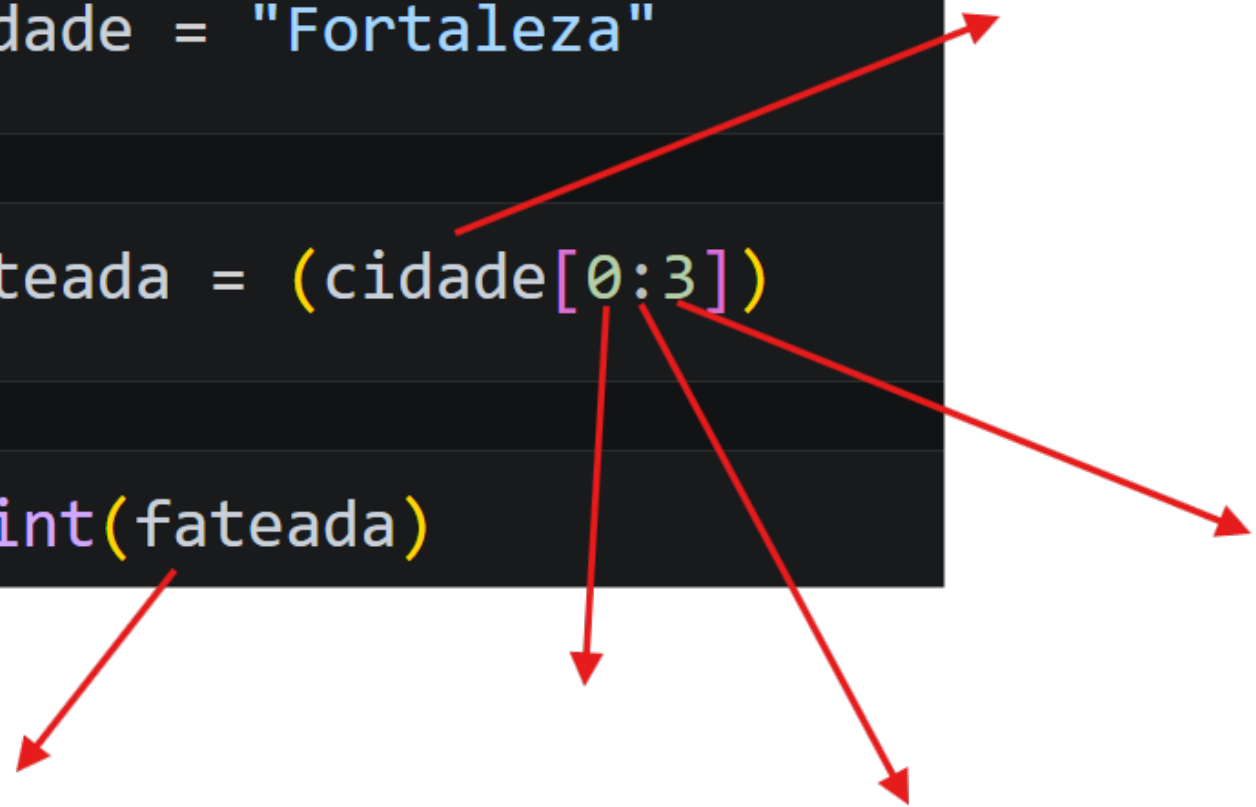
```
cidade = "Fortaleza"
```

✓ 0.0s

```
fateada = (cidade[0:3])
```

✓ 0.0s

```
print(fateada)
```



The diagram consists of five red arrows originating from the code blocks. One arrow points from the string 'Fortaleza' in the first line to the top right. Another points from the slice '[0:3]' in the second line to the middle right. A third points from the variable 'fateada' in the second line to the bottom right. A fourth points from the 'print' function in the third line to the bottom left. The fifth points from the opening parenthesis of the slice in the second line to the bottom center.